Meck Web App on Azure

Azure portfolio project — AZ-104 modules

Author: Andrea Girotto   Subscription: Azure for Students   Region: Norway East

1. Objective

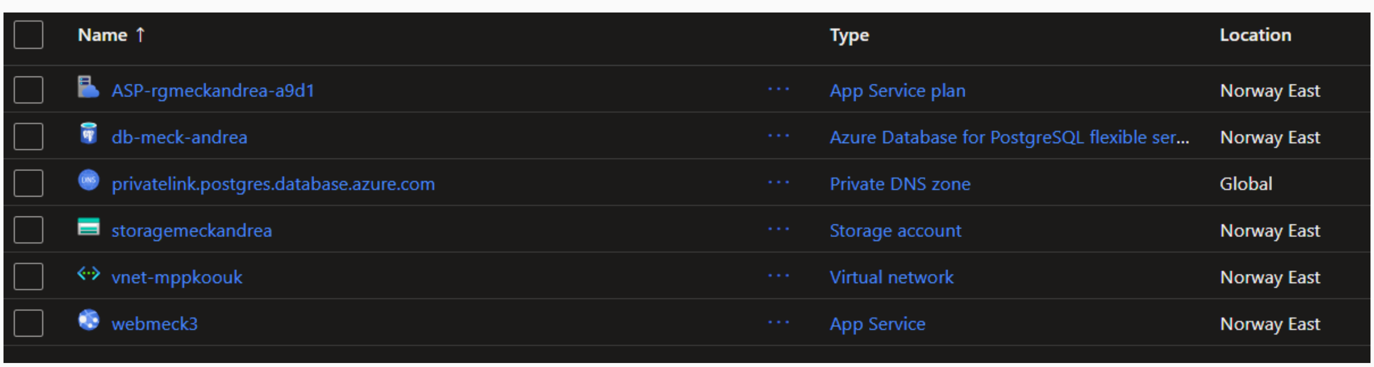
Build a simple web application on Azure that displays a table of flies (meck): the data is read from a PostgreSQL database and each row shows a photo served from Blob Storage. The project is used to demonstrate two AZ-104 modules: deploying and managing compute resources, and managing identities and governance.

2. Architecture

All resources live in a single resource group, rg-meck-andrea. The web app runs on App Service, reaches the PostgreSQL database privately over a virtual network, and reads the fly images from a Blob Storage container.

3. Resources in the resource group

Resource	Type	Purpose
webmeck3	App Service (Linux, Python 3.12)	Hosts the Flask web app
ASP-rgmeckandrea-a9d1	App Service plan (B1)	Compute the web app runs on
db-meck-andrea	Azure Database for PostgreSQL (flexible server)	Stores the meck (flies) table
storagemeckandrea	Storage account (Blob)	Container meck-image with 3 fly photos
vnet-mppkoouk	Virtual network	Private connectivity between app and database
privatelink.postgres…	Private DNS zone	Name resolution for the private database



Name ↑	Type	Location
ASP-rgmeckandrea-a9d1	App Service plan	Norway East
db-meck-andrea	Azure Database for PostgreSQL flexible ser...	Norway East
privatelink.postgres.database.azure.com	Private DNS zone	Global
storagemeckandrea	Storage account	Norway East
vnet-mppkoouk	Virtual network	Norway East
webmeck3	App Service	Norway East

Figure 1 — The six resources of the group in the Azure portal.

4. Deployment

The application code is packaged as a zip and deployed to App Service using the Azure CLI:

```
az webapp deploy \
  --resource-group rg-meck-andrea \
  --name webmeck3 \
  --src-path webmeck3-app.zip \
  --type zip
```

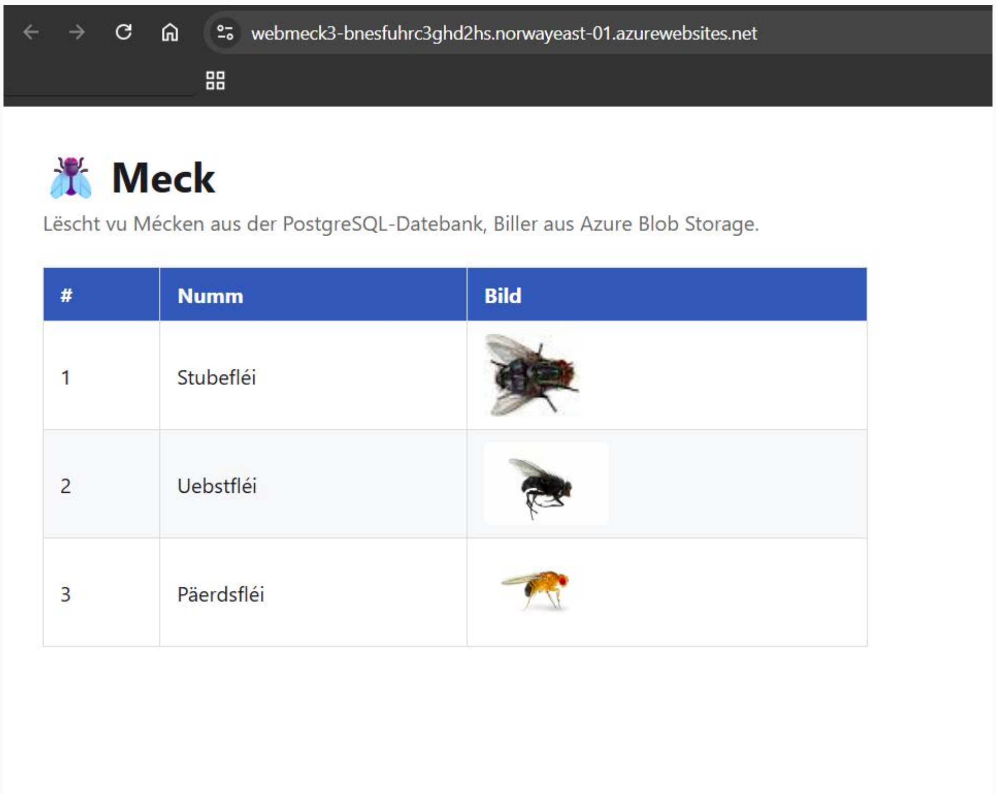


Figure 2 — The deployed application rendering the flies from PostgreSQL and Blob Storage.

5. Key configuration steps

- Deployed the Flask app to App Service via zip deploy (CLI above), with build enabled on deployment.
- Allowed Azure services through the PostgreSQL firewall so the web app can connect.
- Pointed the app at the postgres database; the meck table is created and seeded automatically on first start.

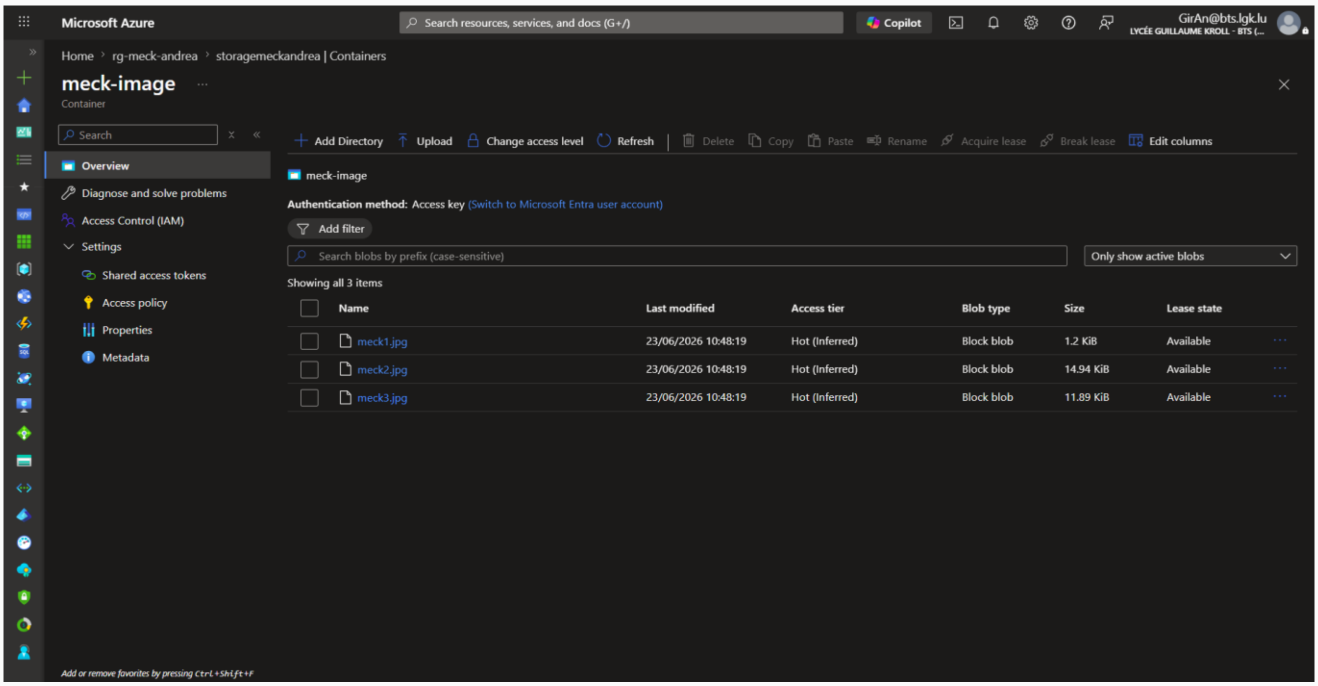


Figure 3 — The three fly images stored in Blob Storage.

6. Azure skills demonstrated

Module	Demonstrated in this project
Deploy and Manage Azure Compute Resources	App Service + App Service plan (B1); zip deployment via Azure CLI; runtime & app-setting configuration; VNet integration; Network Security Group on a dedicated subnet.
Manage Azure Identities and Governance	Resource group organisation; resource tags (owner, team); access control (IAM) at resource-group scope; basis for RBAC and Azure Policy.